ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP

MÔN HỌC: THIẾT KẾ LUẬN LÝ SỐ

ĐỀ TÀI

KIỂM CHỨNG MẠCH TUẦN TỰ ĐẾM TỔNG SỐ BIT 1

Giảng viên hướng dẫn: Hồ Ngọc Diễm
Sinh viên thực hiện: Huỳnh Phạm Ngọc Thắng
Mã số sinh viên: 25521672
Lớp: CE118.R11.VMTN

TP. Hồ Chí Minh, tháng 10 năm 2026

1. GIỚI THIỆU ĐỀ TÀI

1.1. Yêu cầu bài tập

Kiểm chứng mạch tuần tự đếm tổng số bit 1 (One's Counter) trên phần mềm Quartus, với các yêu cầu:

- Trình bày **lưu đồ ASM**, **Datapath** và **Control Unit** của mạch.
- **Vẽ mạch** trên Quartus (Block Diagram/Schematic).
- **Mô phỏng Functional và Timing**, kiểm tra kết quả và **số chu kỳ hoàn thành**.

1.2. Mô tả bài toán

Mạch nhận một số nhị phân 4 bit qua cổng **Inport**, đếm số bit có giá trị 1 và xuất kết quả ra cổng **Outport**. Mạch được thiết kế theo mô hình **FSMD** (Finite State Machine with Datapath) gồm hai phần:

- **Datapath:** register file (R1–R4), ALU, bộ dịch (shifter), mạch kiểm tra Data = 0.
- **Control Unit:** máy trạng thái 8 trạng thái (s0–s7) sinh các tín hiệu điều khiển cho datapath.

Bảng 1.1: Thông số thiết kế

Thông số	Giá trị
Độ rộng dữ liệu (Inport, thanh ghi, ALU)	4 bit
Số thanh ghi	4 (R1–R4) + R0 = hằng số 0
Số trạng thái FSM	8 (s0–s7), mã hóa nhị phân 3 bit $Q_2Q_1Q_0$
Tín hiệu vào	CLK, Start, Inport[3..0]
Tín hiệu ra	Outport[3..0], OE (báo kết quả hợp lệ)
Công cụ	Quartus (Block Diagram + Simulation Waveform Editor)

2. THUẬT TOÁN VÀ LƯU ĐỒ ASM

2.1. Thuật toán đếm bit 1

Ý tưởng: lần lượt lấy bit thấp nhất của Data (bằng phép AND với Mask = 0001), cộng vào biến đếm Ocount, sau đó dịch phải Data 1 bit. Lặp lại cho đến khi Data = 0.

Bảng 2.1: Thuật toán cơ bản của One's Counter

```
1. Data   := Inport
2. Ocount := 0
3. Mask   := 1
   while Data ≠ 0 repeat
4.    Temp   := Data AND Mask
5.    Ocount := Ocount + Temp
6.    Data   := Data >> 1
   end while
7. Outport := Ocount
```

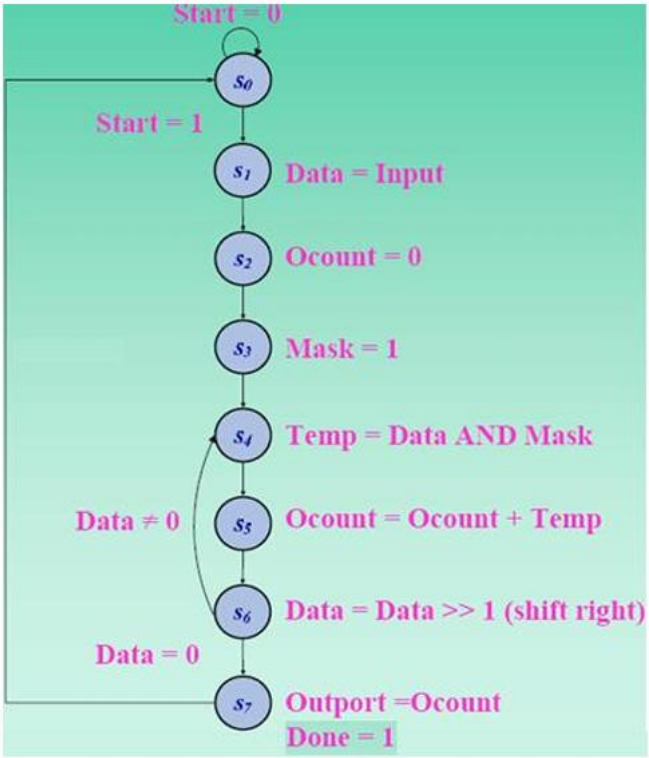
2.2. Gán biến vào thanh ghi

Bảng 2.2: Bảng gán thanh ghi

Thanh ghi	Địa chỉ	Biến	Ý nghĩa
R0	000	0 (hằng số)	Không phải thanh ghi thật, cổng đọc trả về 0000
R1	001	Data	Dữ liệu cần đếm, dịch phải mỗi vòng
R2	010	Mask	Mặt nạ 0001 lấy bit thấp nhất
R3	011	Ocount	Biến đếm số bit 1 (kết quả)
R4	100	Temp	Bit thấp nhất của Data trong vòng hiện tại

2.3. Lưu đồ ASM / Sơ đồ trạng thái

Mỗi bước của thuật toán được gán cho một trạng thái. Mỗi trạng thái thực hiện một phép toán thanh ghi (RTL) trong một chu kỳ clock.



Hình 2.1: Lưu đồ ASM (sơ đồ trạng thái) của One's Counter

Bảng 2.3: Mô tả các trạng thái

Trạng thái	$Q_2Q_1Q_0$	Thao tác RTL	Trạng thái kế tiếp
s0	000	Chờ	Start = 1 → s1; Start = 0 → s0
s1	001	Data := Inport	s2
s2	010	Ocount := 0	s3
s3	011	Mask := 1	s4
s4	100	Temp := Data AND Mask	s5
s5	101	Ocount := Ocount + Temp	s6
s6	110	Data := Data >> 1	Data ≠ 0 → s4; Data = 0 → s7
s7	111	Outport := Ocount, Done = 1 (OE = 1)	s0

3. THIẾT KẾ DATAPATH

3.1. Cấu trúc tổng quát

Datapath gồm các khối nối theo chuỗi: **Register file → cổng đọc A/B → ALU → Shifter** → ghi ngược vào register file. Riêng R1 có thêm đường nạp từ Inport (điều khiển bởi IE). Tín hiệu **DATA_ZERO** lấy từ R1 gửi về Control Unit.

3.2. Bảng từ điều khiển (Control Words)

Bảng 3.1: Control words cho One's Counter

CW	IE	Write addr	Read addr A	Read addr B	ALU	Shifter	OE
1	1	R1	X	X	X	X	0
2	0	R3	0	0	Add	Pass	0
3	0	R2	0	X	Increment	Pass	0
4	0	R4	R1	R2	AND	Pass	0
5	0	R3	R3	R4	Add	Pass	0
6	0	R1	R1	0	Add	Shift right	0
7	0	None	R3	0	Add	Pass	1

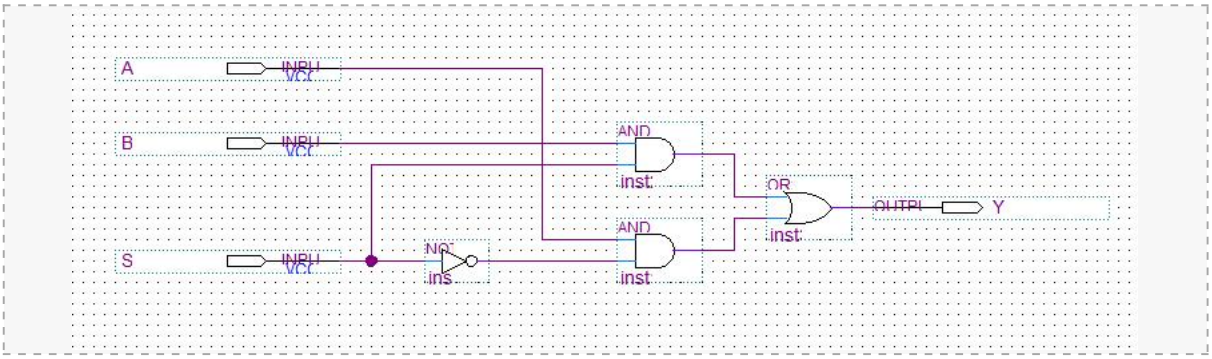
Control word 4, 5, 6 được lặp lại khi Data ≠ 0.

3.3. Các khối cơ bản

3.3.1. MUX 2-1

$$Y = S' \cdot A + S \cdot B$$

S = 0 → Y = A; S = 1 → Y = B. Khối này được dùng trong thanh ghi, bộ chọn D_R1, cổng đọc và bộ dịch.



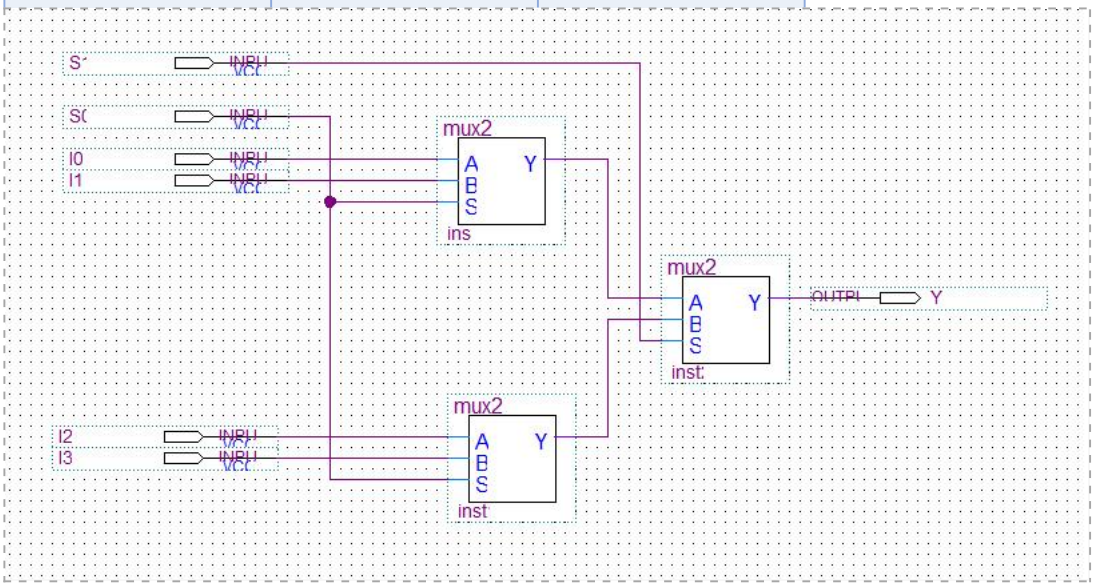
Hình 3.1: Mạch MUX 2-1

3.3.2. MUX 4-1

Ghép từ 3 MUX 2-1: tầng 1 chọn theo S0, tầng 2 chọn theo S1.

Bảng 3.2: Bảng chọn MUX 4-1

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3



Hình 3.2: Mạch MUX 4-1

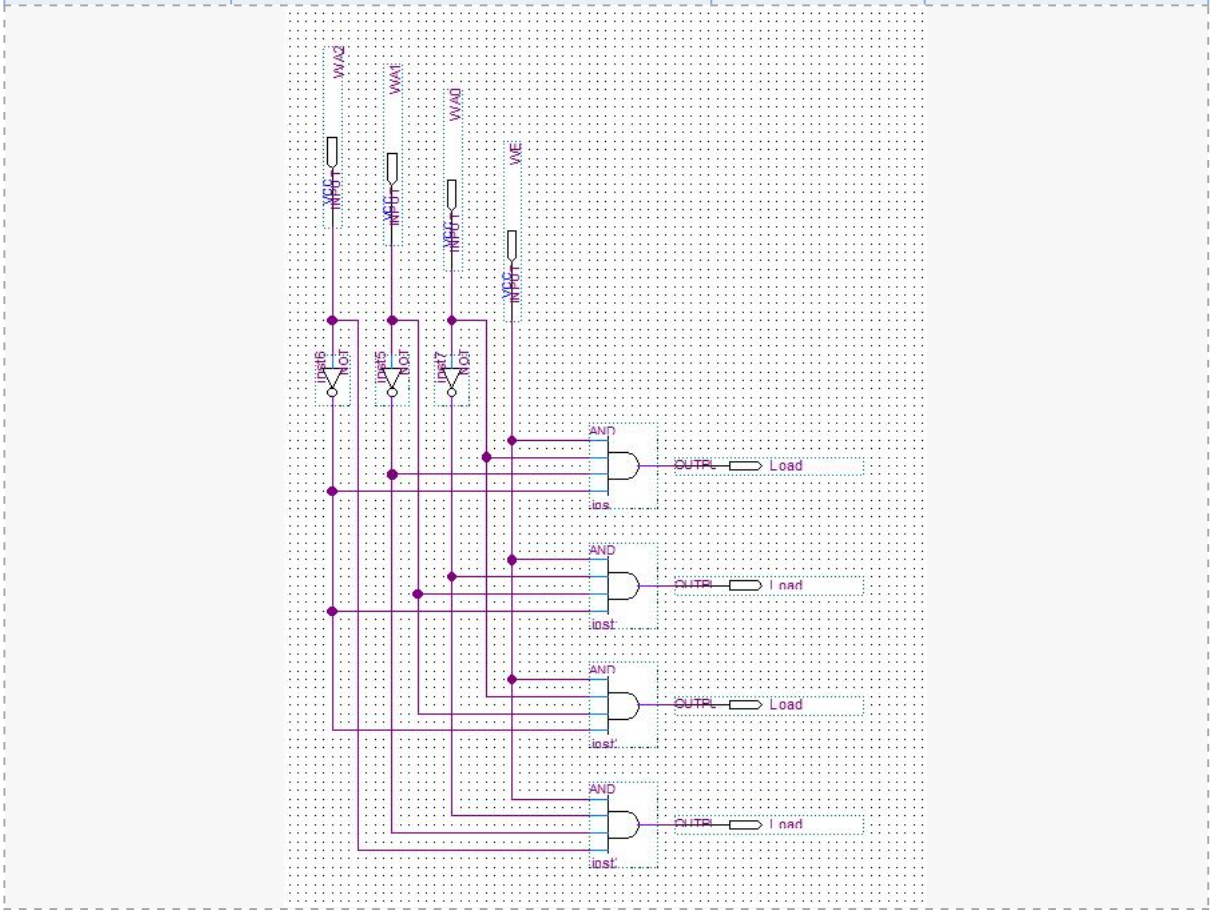
3.4. Register File

3.4.1. Bộ giải mã địa chỉ ghi (Load_W)

Chọn thanh ghi được ghi dựa trên địa chỉ $WA_2WA_1WA_0$ và cho phép ghi WE:

Bảng 3.3: Phương trình tín hiệu Load

Tín hiệu	Phương trình	Địa chỉ	Thanh ghi
Load1	$WE \cdot WA_2' \cdot WA_1' \cdot WA_0$	001	R1 (Data)
Load2	$WE \cdot WA_2' \cdot WA_1 \cdot WA_0'$	010	R2 (Mask)
Load3	$WE \cdot WA_2' \cdot WA_1 \cdot WA_0$	011	R3 (Ocount)
Load4	$WE \cdot WA_2 \cdot WA_1' \cdot WA_0'$	100	R4 (Temp)



Hình 3.3: Mạch giải mã ghi Load_W (Load1–Load4)

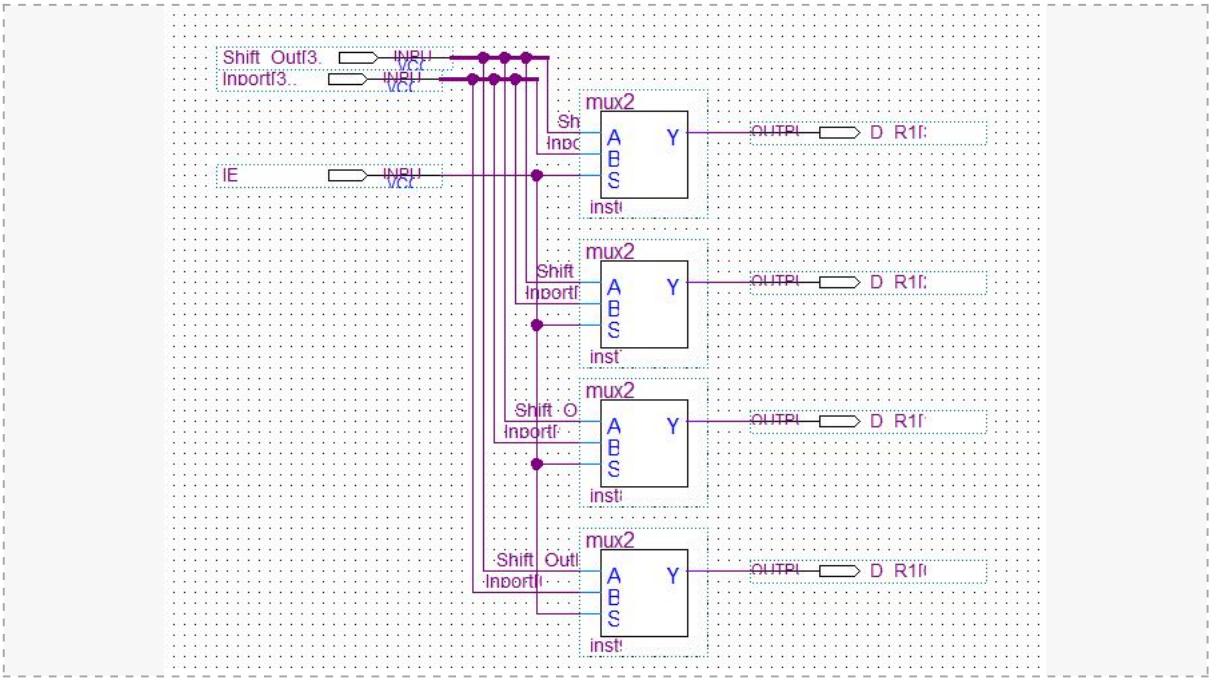
3.4.2. Chọn dữ liệu ghi D_R1 và D_R

R1 có thể nhận dữ liệu từ Inport (ở s1) hoặc từ bộ dịch (ở s6), nên cần MUX 2-1 điều khiển bởi IE:

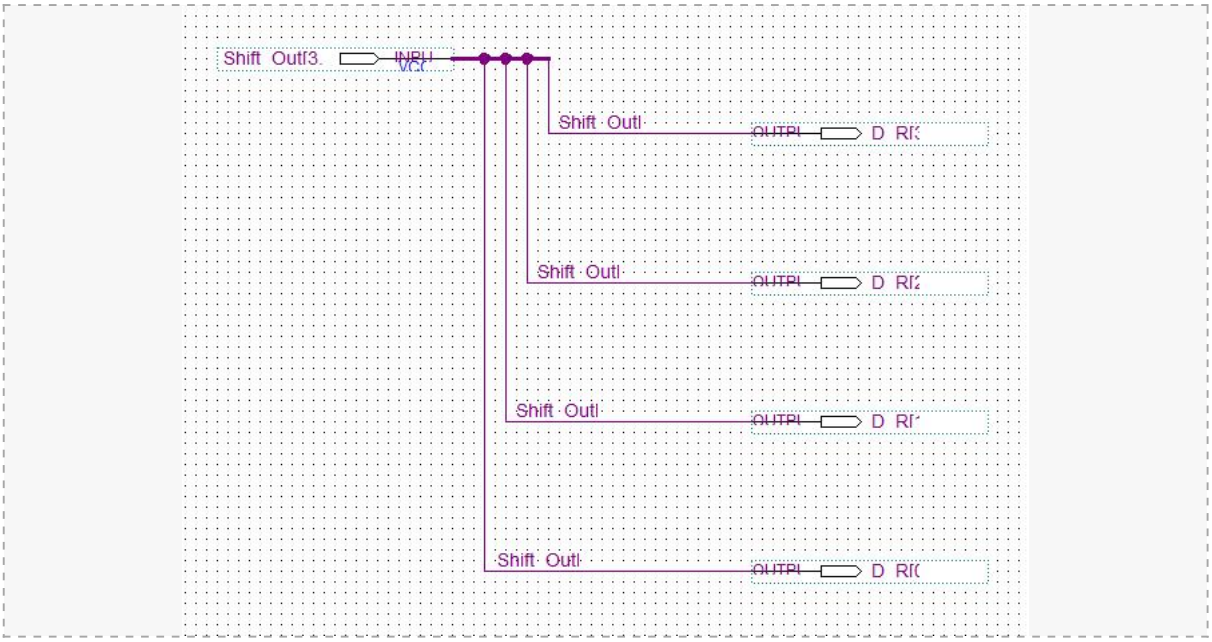
$$D_R1[i] = IE' \cdot Shift_Out[i] + IE \cdot Inport[i] \quad (i = 3..0)$$

R2, R3, R4 chỉ nhận dữ liệu từ bộ dịch:

$$D_R[i] = Shift_Out[i]$$



Hình 3.4: Mạch chọn dữ liệu ghi D_R1

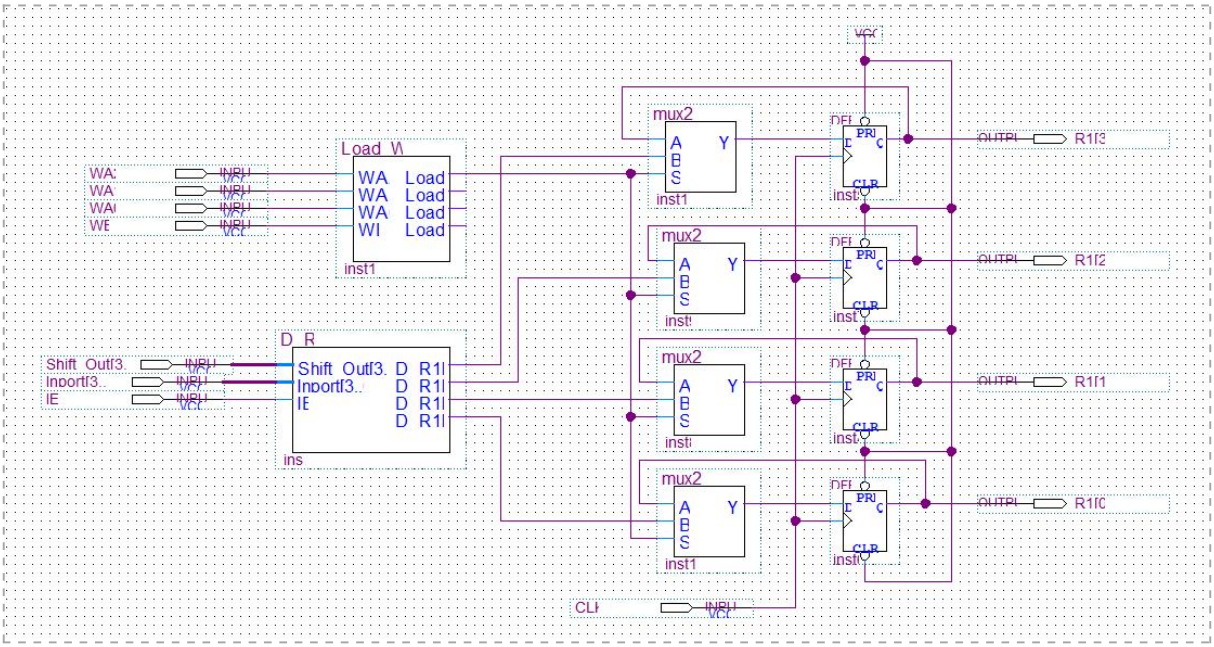


Hình 3.5: Mạch D_R (dùng chung cho R2, R3, R4)

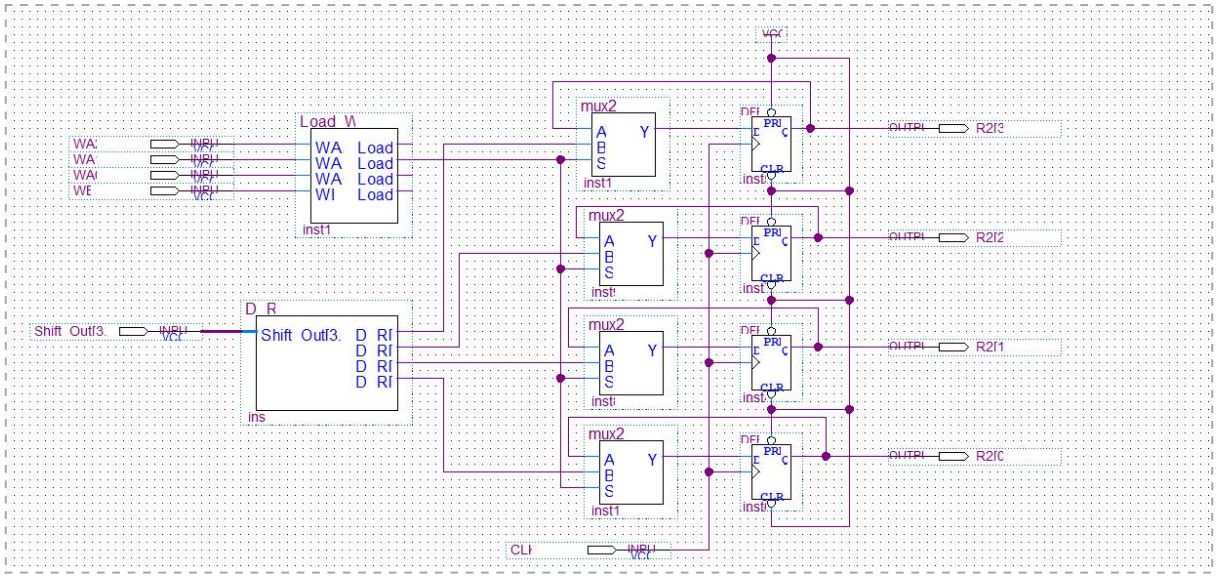
3.4.3. Thanh ghi 4 bit có tín hiệu Load

Mỗi bit gồm một MUX 2-1 và một D flip-flop: A = Q (giữ giá trị), B = dữ liệu mới, S = Load. PRN và CLRN nối VCC. Khi Load = 1 thanh ghi nạp dữ liệu mới tại cạnh lên CLK, khi Load = 0 thanh ghi giữ nguyên giá trị.

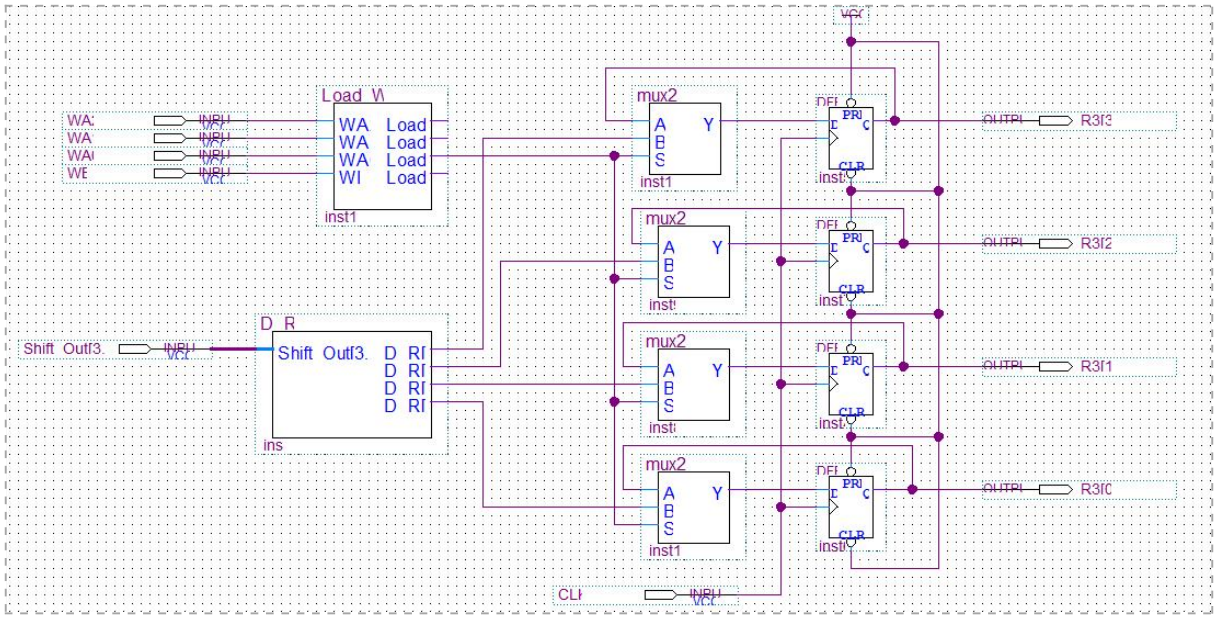
Q[i](next) = Load'·Q[i] + Load·D[i]



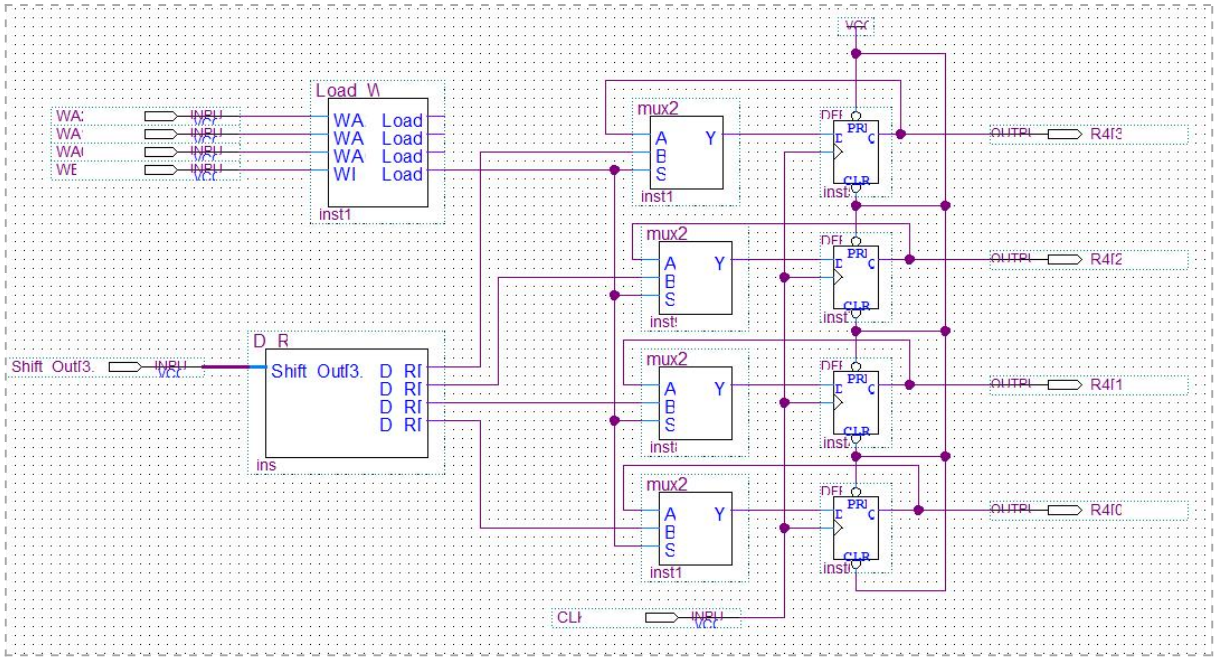
Hình 3.6: Thanh ghi R1 (Data)



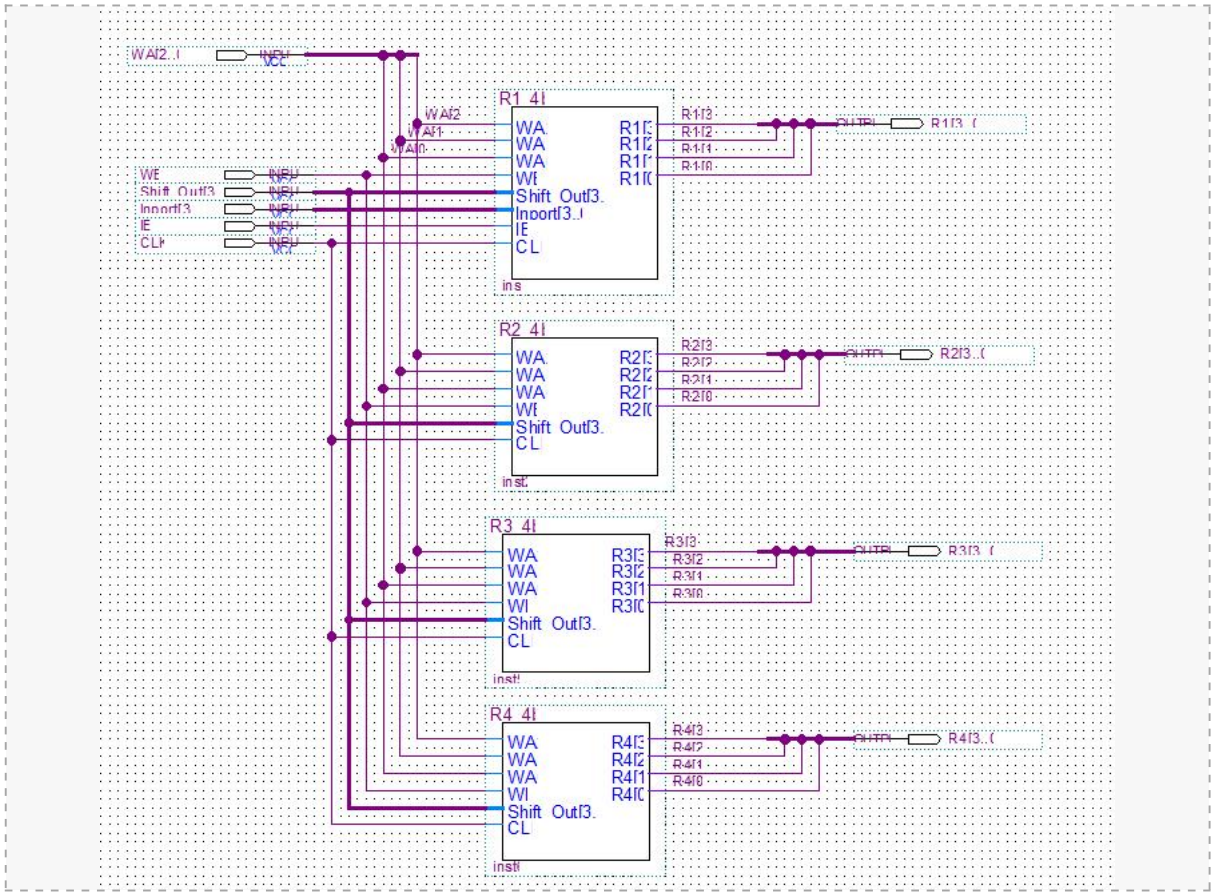
Hình 3.7: Thanh ghi R2 (Mask)



Hình 3.8: Thanh ghi R3 (Ocount)



Hình 3.9: Thanh ghi R4 (Temp)



Hình 3.10: Register file gộp (register_4)

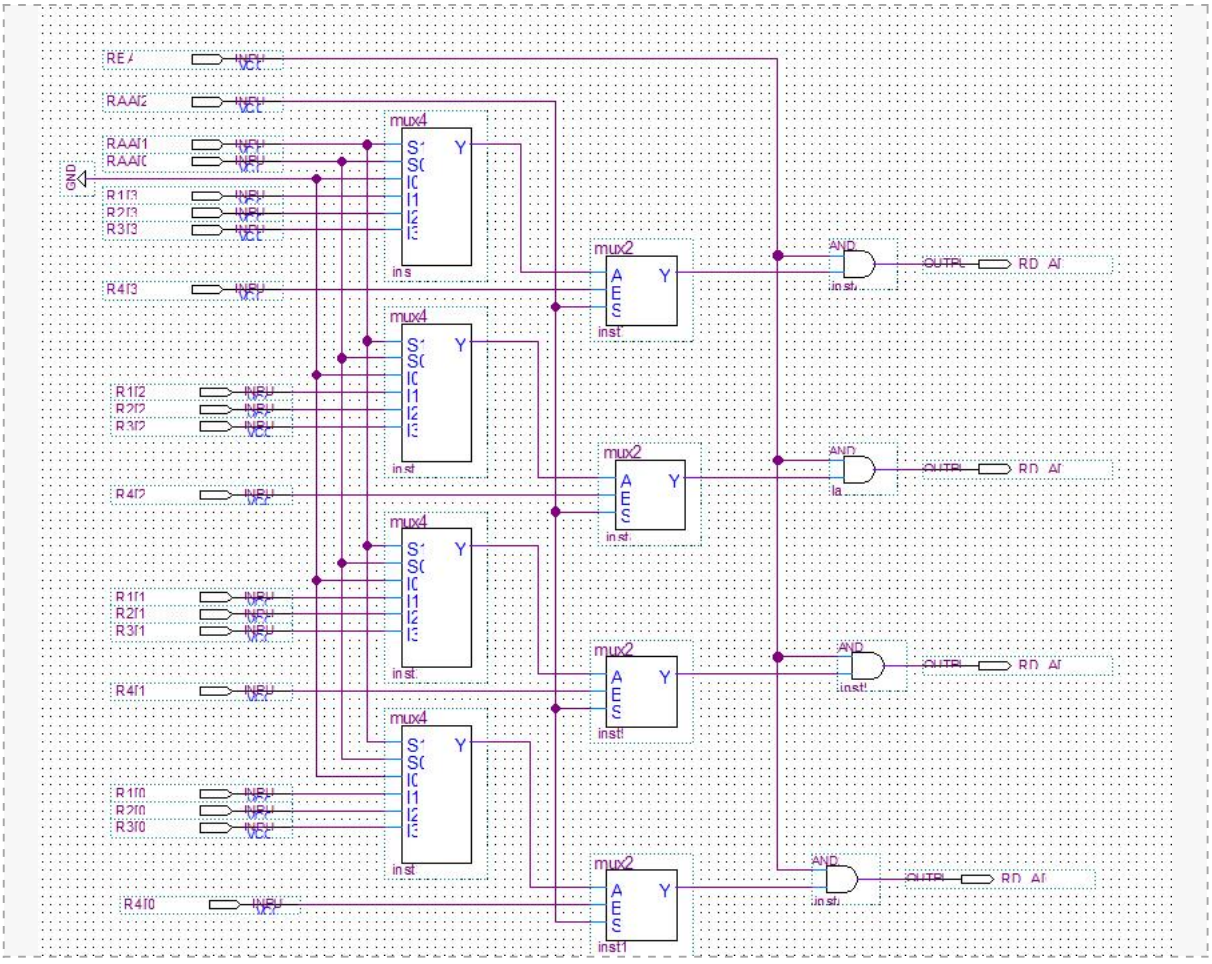
3.5. Cổng đọc Read A / Read B

Mỗi bit dùng MUX 4-1 (S1S0 = RA_1RA_0) chọn giữa 0, R1, R2, R3; sau đó MUX 2-1 (S = RA_2) chọn thêm R4; cuối cùng AND với tín hiệu cho phép đọc RE.

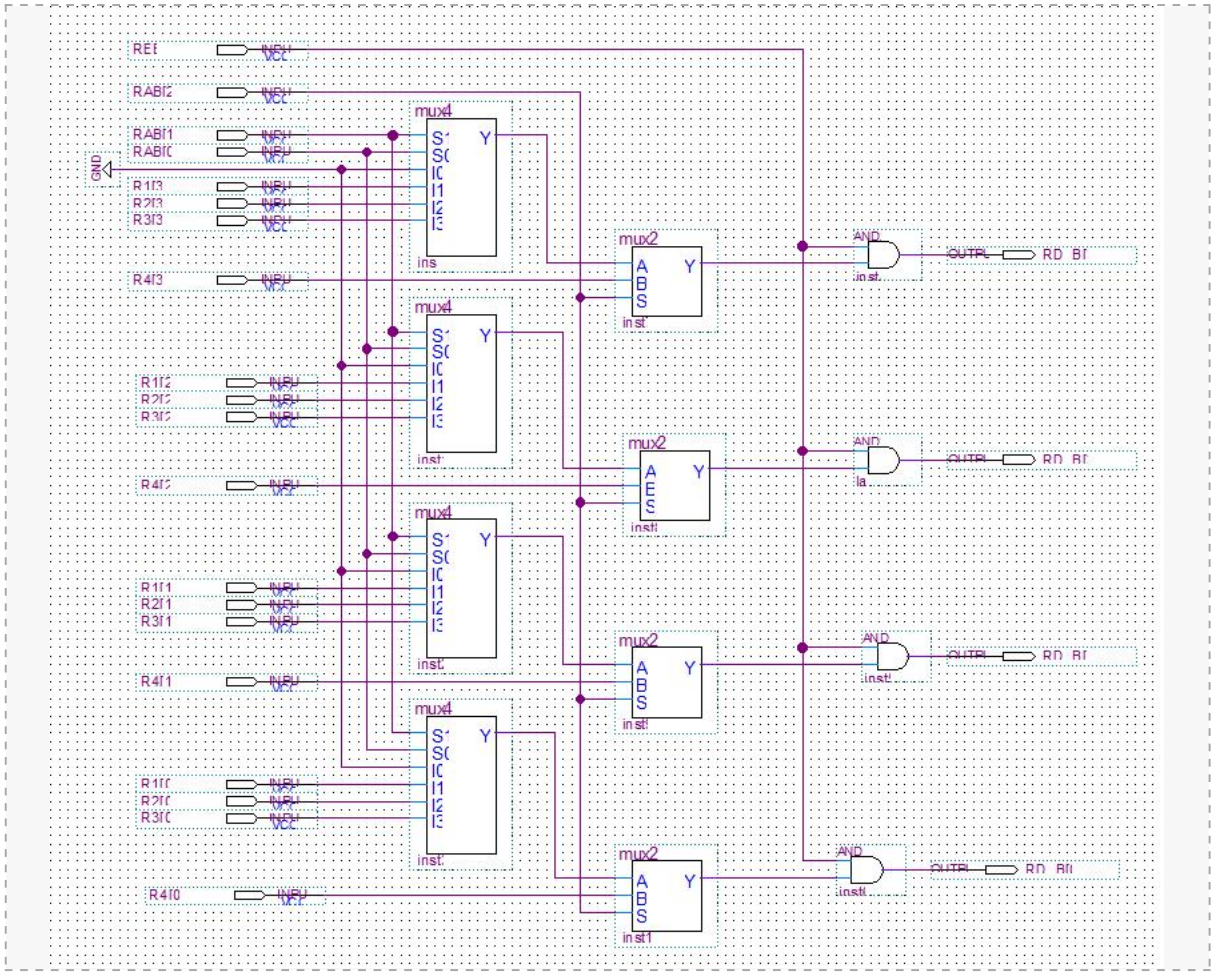
Bảng 3.4: Địa chỉ đọc

$RA_2RA_1RA_0$	Đầu ra RD	Nguồn
000	0000	R0 (hằng 0, từ GND)
001	R1	Data
010	R2	Mask
011	R3	Ocount
100	R4	Temp
RE = 0	0000	Không đọc

RD_A[i] = REA · MUX2(MUX4(0, R1[i], R2[i], R3[i]; RAA_1RAA_0), R4[i]; RAA_2)



Hình 3.11: Cổng đọc Read A



Hình 3.12: Cổng đọc Read B

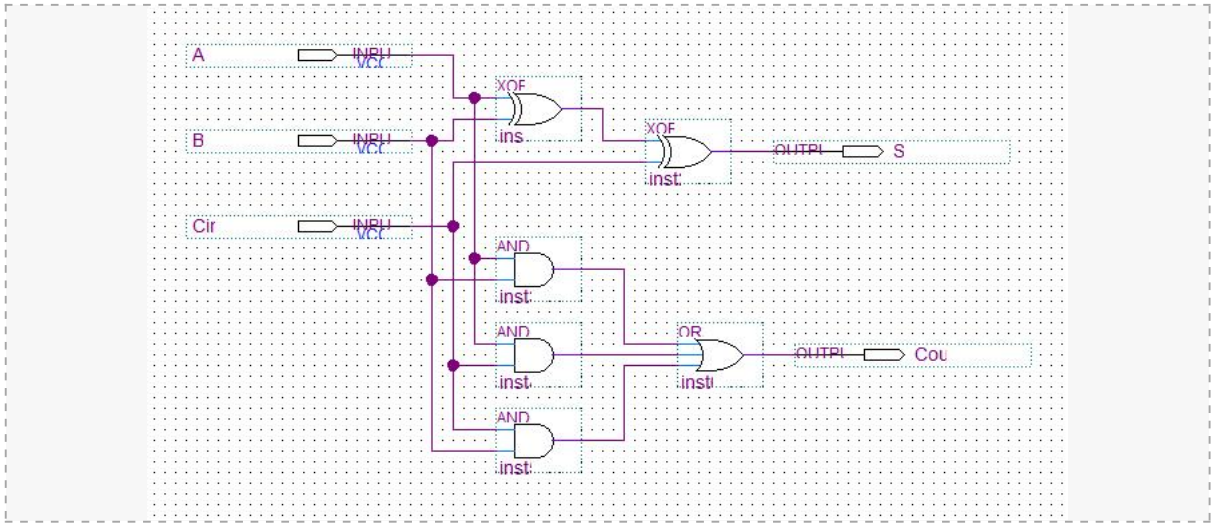
3.6. ALU

3.6.1. Full adder và bộ cộng 4 bit

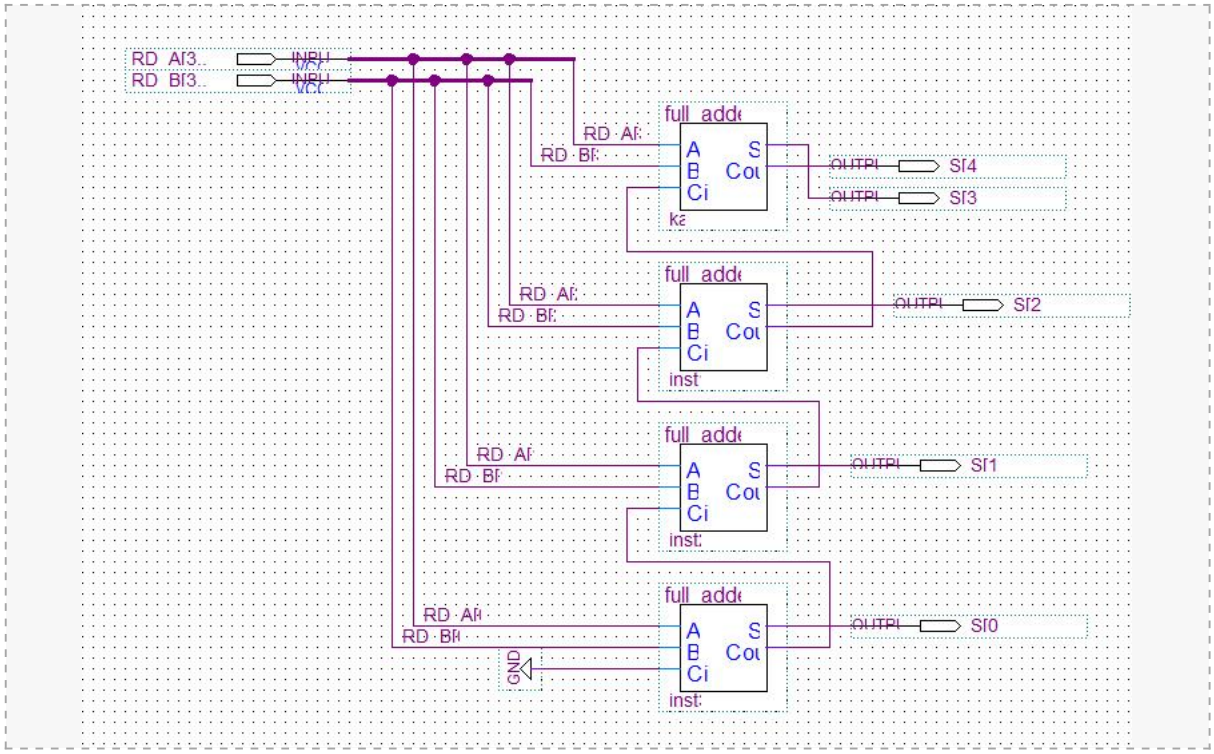
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + A \cdot C_{in} + B \cdot C_{in}$$

Bộ cộng 4 bit ghép nối tiếp (ripple carry) 4 full adder, C_{in} của bit 0 nối GND, C_{out} bit 3 là S[4] (không dùng).



Hình 3.13: Mạch Full adder

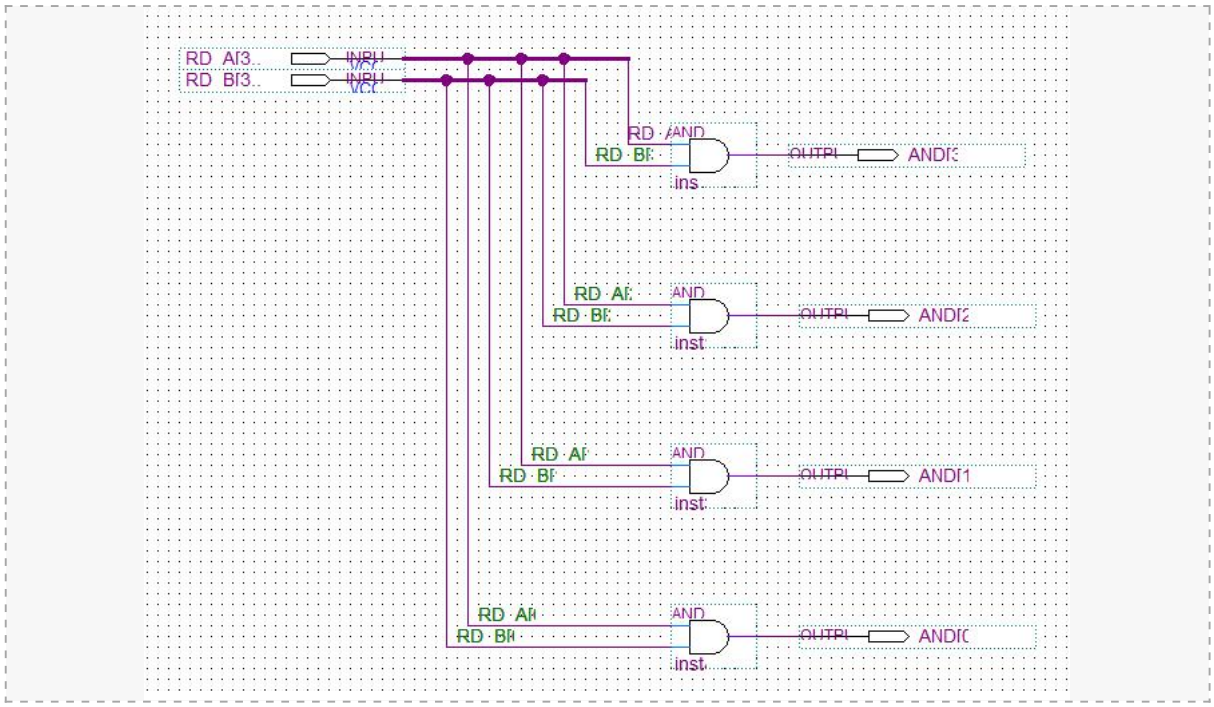


Hình 3.14: Bộ cộng 4 bit (full_adder_4)

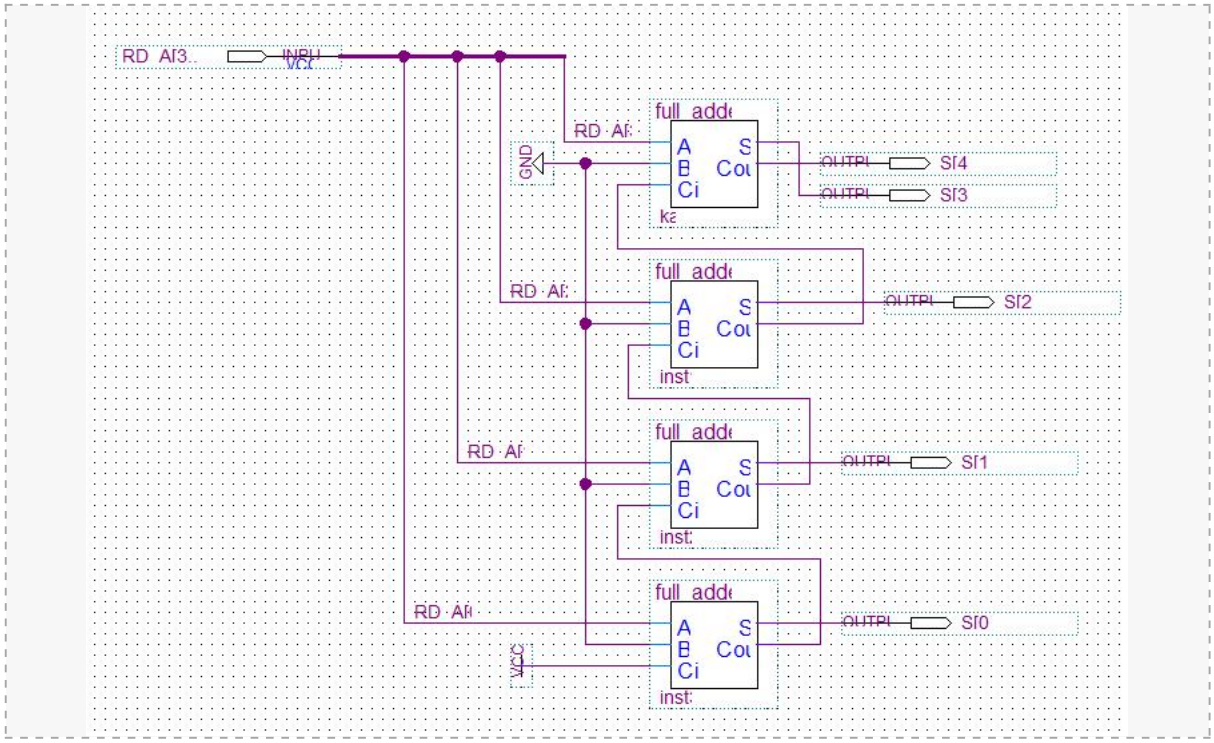
3.6.2. Khối AND 4 bit và khối Increment

$$AND[i] = RD_A[i] \cdot RD_B[i]$$

$$INC = RD_A + 1 \quad (B = 0000, C_{in} \text{ bit } 0 = VCC)$$



Hình 3.15: Khối AND 4 bit



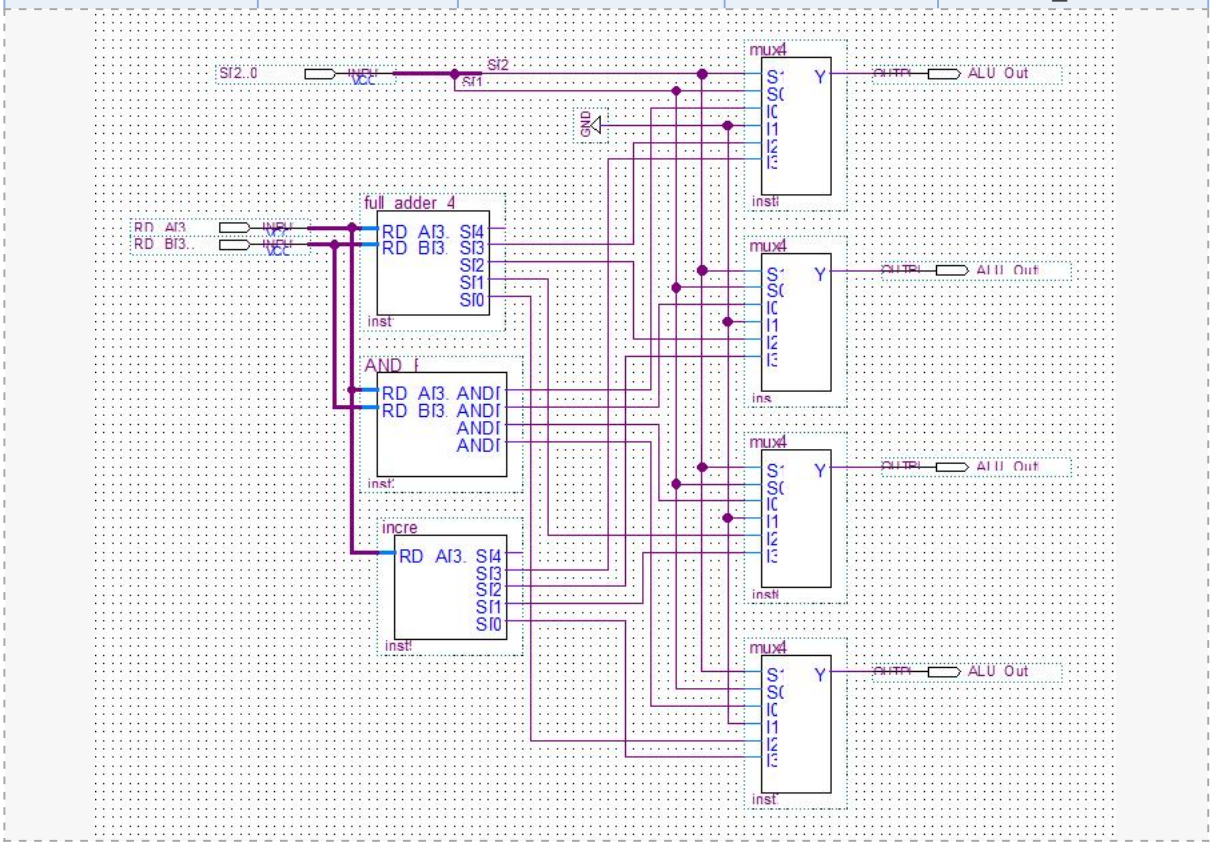
Hình 3.16: Khối Increment (incre)

3.6.3. Bộ chọn phép toán ALU

Vì S_0 luôn bằng 1 ở cả 3 phép, ALU chỉ cần dùng S_2S_1 làm tín hiệu chọn của MUX 4-1 cho mỗi bit:

Bảng 3.5: Mã phép toán ALU

Phép toán	$S_2S_1S_0$	S_2S_1 (chọn MUX)	Ngõ vào MUX	ALU_Out
AND	001	00	I0	RD_A AND RD_B
(không dùng)	011	01	I1	0000 (GND)
Add	101	10	I2	RD_A + RD_B
Increment	111	11	I3	RD_A + 1



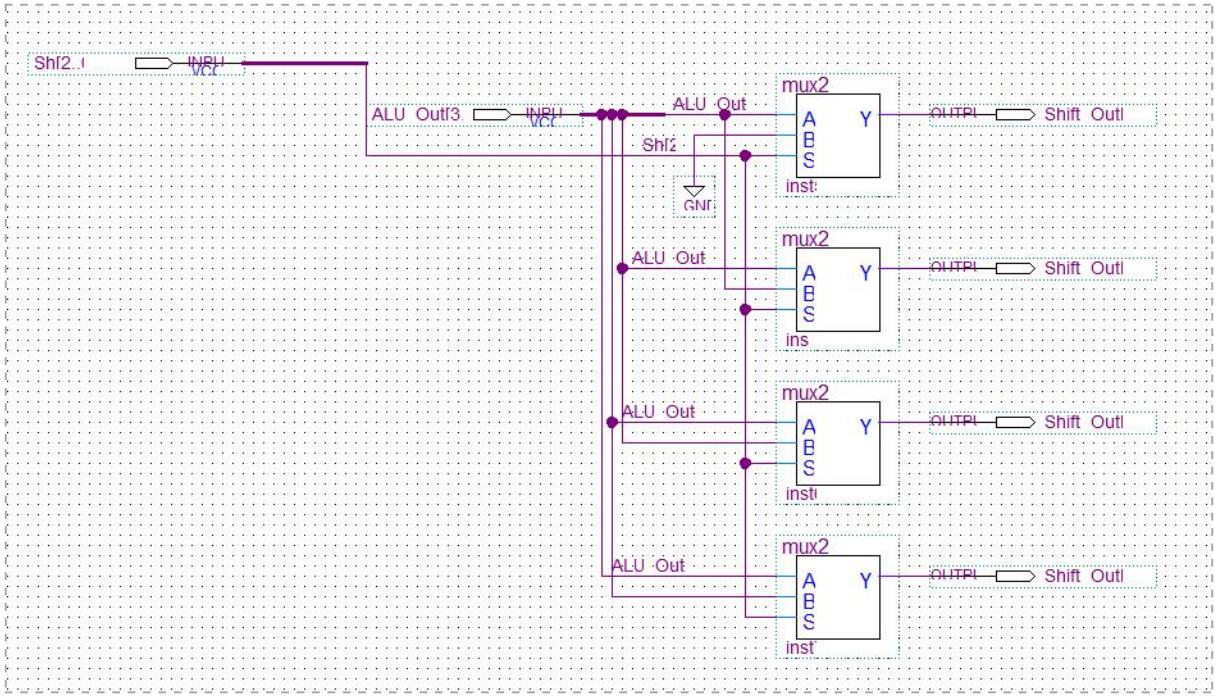
Hình 3.17: Khối ALU

3.7. Bộ dịch (Shifter)

Bảng 3.6: Chức năng bộ dịch

$Sh_2Sh_1Sh_0$	Chức năng	Shift_Out[3]	Shift_Out[2]	Shift_Out[1]	Shift_Out[0]
000	Pass	ALU_Out[3]	ALU_Out[2]	ALU_Out[1]	ALU_Out[0]
110	Shift right	0	ALU_Out[3]	ALU_Out[2]	ALU_Out[1]

Mỗi bit dùng MUX 2-1 với S = Sh_2 (vì chỉ có 2 chế độ): A = ALU_Out[i], B = ALU_Out[i+1] (bit 3 nhận GND).

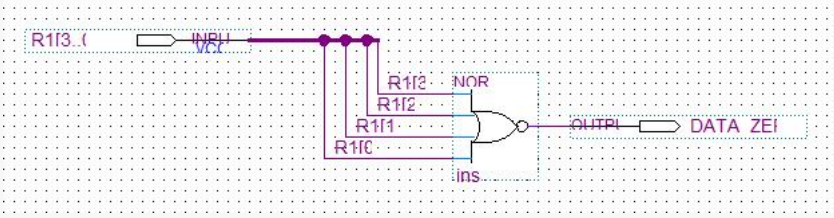


Hình 3.18: Bộ dịch (Shifter)

3.8. Mạch kiểm tra Data = 0

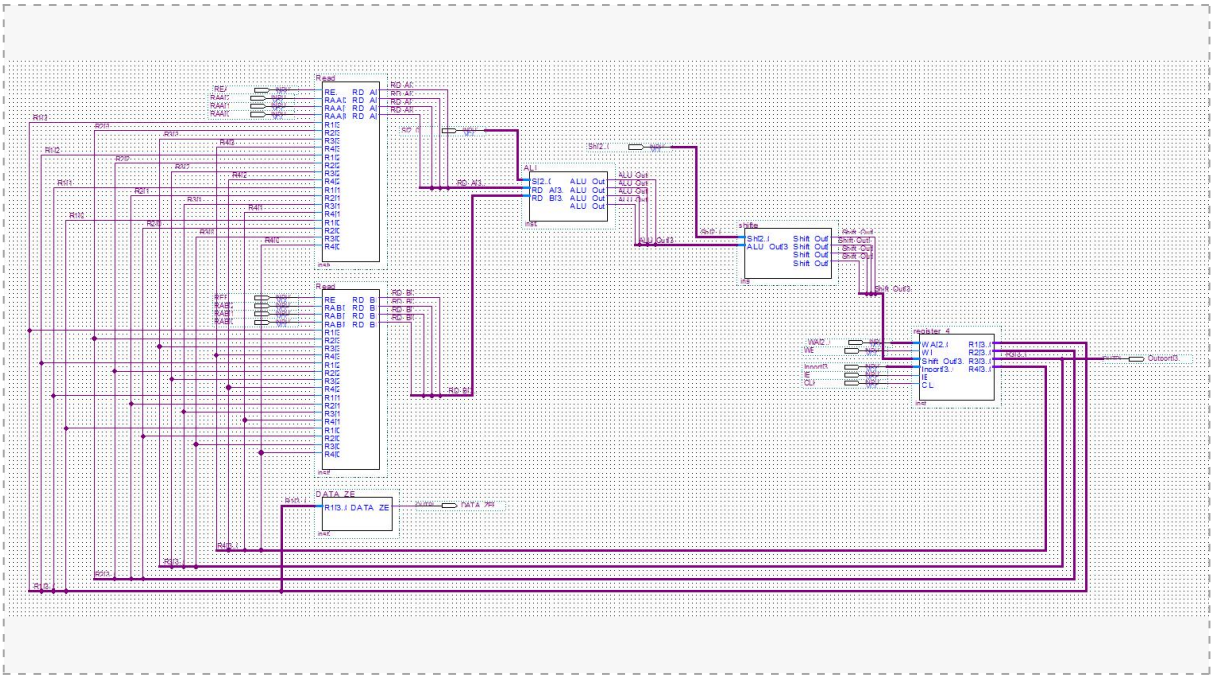
DATA_ZERO = (R1[3] + R1[2] + R1[1] + R1[0])'

Dùng cổng NOR 4 ngõ vào, đầu vào lấy từ thanh ghi R1 (Data). Tín hiệu này gửi về Control Unit để quyết định thoát vòng lặp ở s6.



Hình 3.19: Mạch DATA_ZERO

3.9. Datapath hoàn chỉnh



Hình 3.20: Datapath hoàn chỉnh

4. THIẾT KẾ CONTROL UNIT

Control Unit là máy trạng thái Moore gồm 3 phần: **thanh ghi trạng thái** (3 D flip-flop), **logic trạng thái kế tiếp** (next-state logic) và **logic ngõ ra** (output logic). Ngõ vào là Start và DATA_ZERO.

4.1. Bảng trạng thái kế tiếp

Bảng 4.1: Bảng trạng thái kế tiếp $Q_2Q_1Q_0$(next)

Trạng thái	$Q_2Q_1Q_0$	Start, (Data = 0)			
		00	01	10	11
s0	000	000	000	001	001
s1	001	010	010	010	010
s2	010	011	011	011	011
s3	011	100	100	100	100
s4	100	101	101	101	101
s5	101	110	110	110	110
s6	110	100	111	100	111
s7	111	000	000	000	000

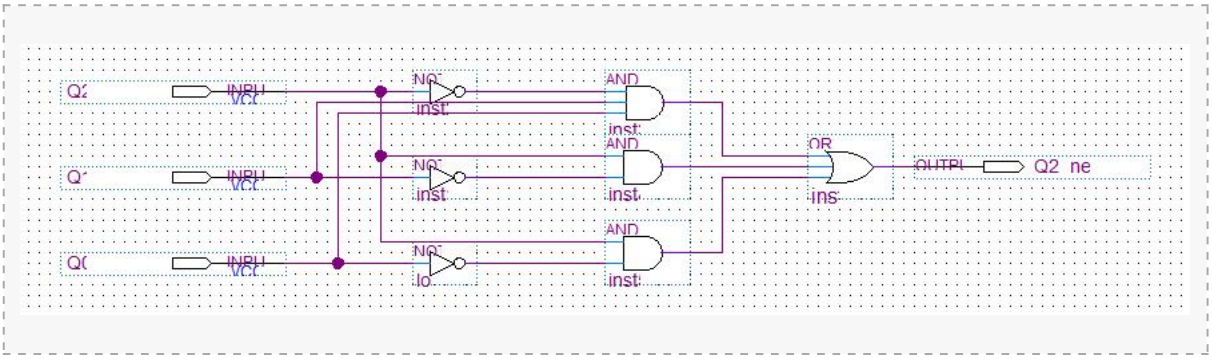
4.2. Phương trình trạng thái kế tiếp

$$Q_2(next) = Q_2'Q_1Q_0 + Q_2Q_1' + Q_2Q_0'$$

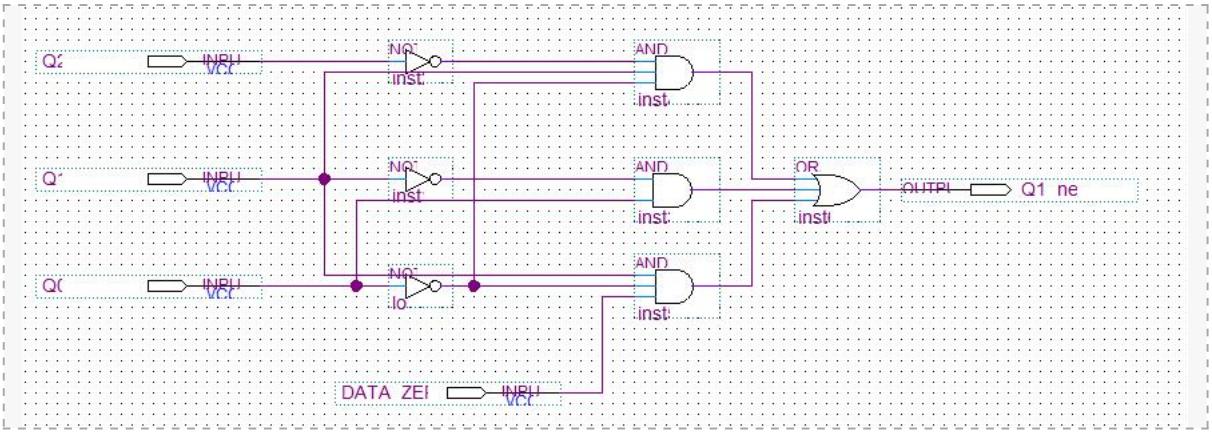
$$Q_1(next) = Q_1'Q_0 + Q_2'Q_1Q_0' + (Data=0)\cdot Q_1Q_0'$$

$$Q_0(next) = Q_2'Q_1Q_0' + Q_2Q_1'Q_0' + Start\cdot Q_2'Q_0' + (Data=0)\cdot Q_2Q_0'$$

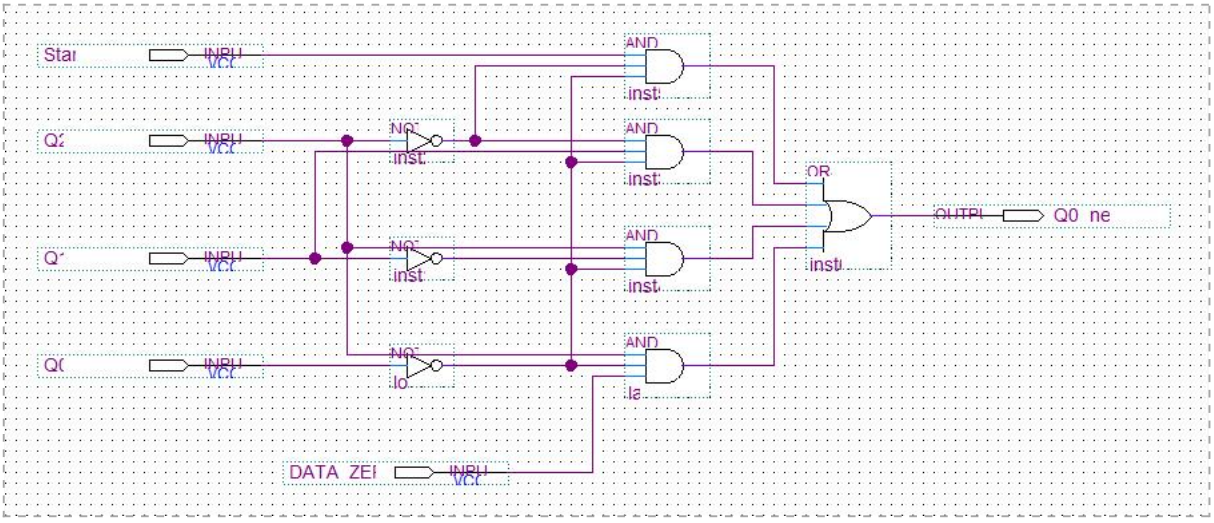
Ghi chú: Q_2(next) bằng 1 tại s3, s4, s5, s6 bất kể Start và Data = 0, nên không phụ thuộc (Data = 0). Phương trình có thể rút gọn thành $\mathbf{Q_2(next) = Q_2 \oplus Q_1Q_0}$.



Hình 4.2: Mạch Q2_next



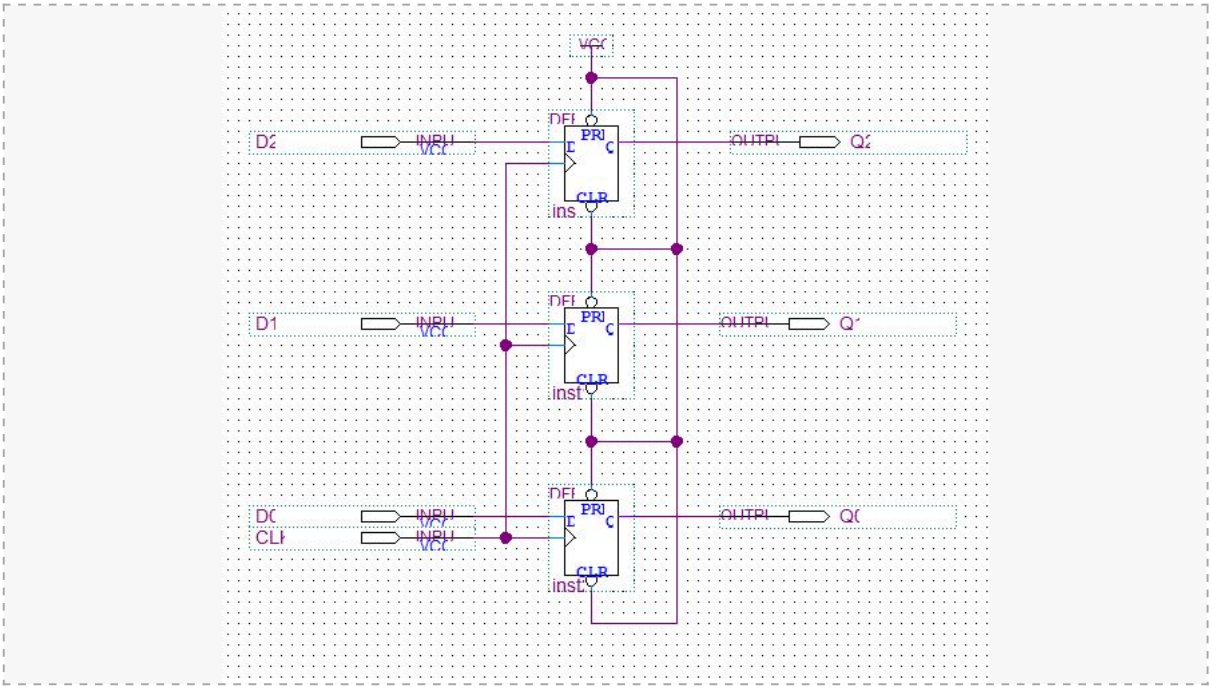
Hình 4.3: Mạch Q1_next



Hình 4.4: Mạch Q0_next

4.3. Thanh ghi trạng thái

Gồm 3 D flip-flop dùng chung CLK, PRN và CLRN nối VCC. Đầu vào $D_2D_1D_0$ = $Q_2Q_1Q_0$(next), đầu ra $Q_2Q_1Q_0$ đưa đến output logic và hồi tiếp về next-state logic.



Hình 4.5: Thanh ghi trạng thái (current_state)

4.4. Bảng ngõ ra

Bảng 4.2: Bảng ngõ ra – phần 1 (ghi và đọc A)

State	$Q_2Q_1Q_0$	IE	WA_2	WA_1	WA_0	WE	RAA_2	RAA_1	RAA_0	REA
s0	000	0	X	X	X	0	X	X	X	0
s1	001	1	0	0	1	1	X	X	X	0
s2	010	0	0	1	1	1	0	0	0	1
s3	011	0	0	1	0	1	0	0	0	1
s4	100	0	1	0	0	1	0	0	1	1
s5	101	0	0	1	1	1	0	1	1	1
s6	110	0	0	0	1	1	0	0	1	1
s7	111	0	X	X	X	0	0	1	1	1

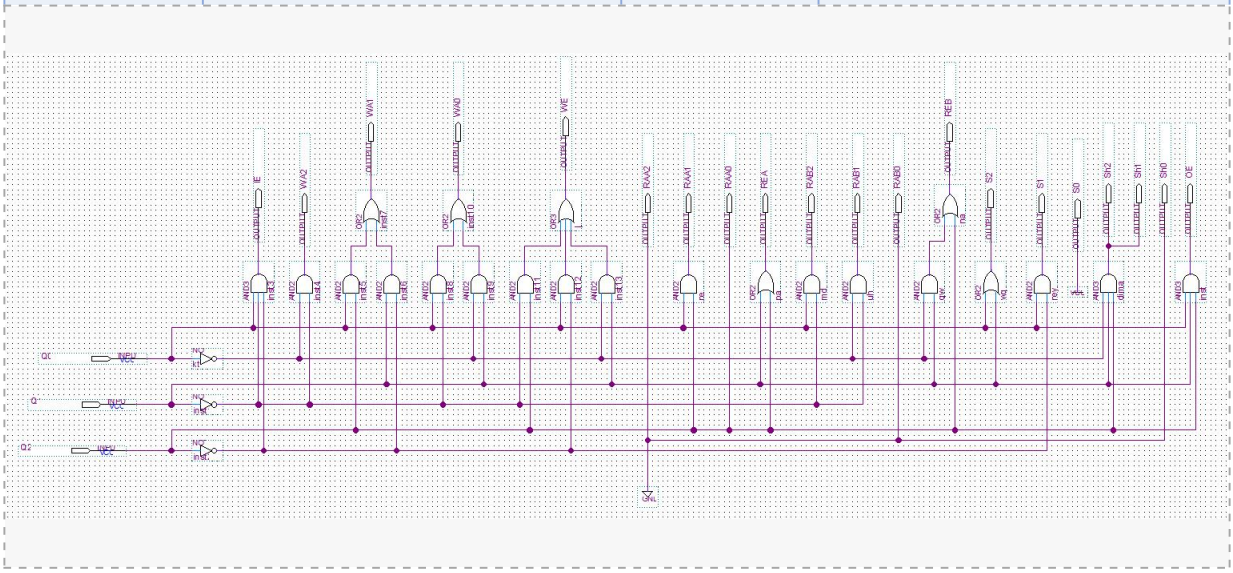
Bảng 4.3: Bảng ngõ ra – phần 2 (đọc B, ALU, Shifter, OE)

State	RAB_2	RAB_1	RAB_0	REB	S_2	S_1	S_0	Sh_2	Sh_1	Sh_0	OE
s0	X	X	X	0	X	X	X	X	X	X	0
s1	X	X	X	0	X	X	X	X	X	X	0
s2	0	0	0	1	1	0	1	0	0	0	0
s3	X	X	X	0	1	1	1	0	0	0	0
s4	0	1	0	1	0	0	1	0	0	0	0
s5	1	0	0	1	1	0	1	0	0	0	0
s6	0	0	0	1	1	0	1	1	1	0	0
s7	0	0	0	1	1	0	1	0	0	0	1

4.5. Phương trình ngõ ra

Bảng 4.4: Phương trình logic ngõ ra

Tín hiệu	Phương trình	Tín hiệu	Phương trình
IE	$Q_2'Q_1'Q_0$	RAB_2	$Q_1'Q_0$
WA_2	$Q_1'Q_0'$	RAB_1	$Q_1'Q_0'$
WA_1	$Q_2'Q_1 + Q_2Q_0$	RAB_0	0
WA_0	$Q_1'Q_0 + Q_1Q_0'$	REB	$Q_2 + Q_1Q_0'$
WE	$Q_2'Q_0 + Q_1Q_0' + Q_2Q_1'$	S_2	$Q_1 + Q_0$
RAA_2	0	S_1	$Q_2'Q_0$
RAA_1	Q_2Q_0	S_0	1
RAA_0	Q_2	$Sh_2 = Sh_1$	$Q_2Q_1Q_0'$
REA	$Q_2 + Q_1$	Sh_0	0
		OE	$Q_2Q_1Q_0$



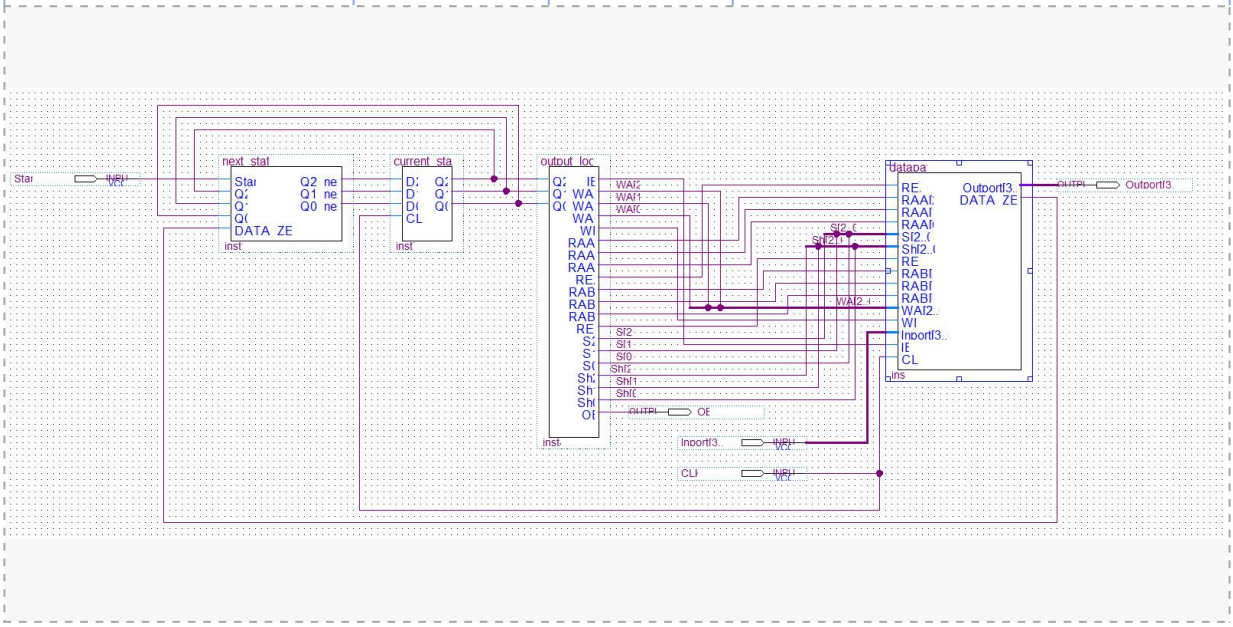
Hình 4.6: Mạch logic ngõ ra (output_logic)

5. MẠCH TỔNG HOÀN CHỈNH

Mạch tổng ghép Control Unit (next_state, current_state, output_logic) với Datapath. $Q_2Q_1Q_0$ từ thanh ghi trạng thái đưa vào output logic và hồi tiếp về next-state logic; DATA_ZERO từ datapath đưa về next-state logic; CLK dùng chung cho thanh ghi trạng thái và datapath.

Bảng 5.1: Các chân vào/ra của mạch tổng

Chân	Hướng	Độ rộng	Ý nghĩa
CLK	Vào	1	Xung clock hệ thống
Start	Vào	1	Bắt đầu đếm (đọc ở s0)
Inport[3..0]	Vào	4	Dữ liệu cần đếm, nạp vào R1 ở s1
Outport[3..0]	Ra	4	Giá trị Ocount (R3)
OE	Ra	1	Đèn báo kết quả hợp lệ (= 1 ở s7)



Hình 5.1: Mạch tổng One's Counter (Control Unit + Datapath)

Về OE: OE được dùng làm tín hiệu báo kết quả hợp lệ. Outport hiển thị giá trị Ocount trong suốt quá trình đếm; giá trị đọc tại chu kỳ OE = 1 là kết quả cuối cùng.

6. MÔ PHỎNG VÀ KIỂM CHỨNG

6.1. Thiết lập mô phỏng

- CLK chu kỳ **T = 20 ns** (cạnh lên tại 10, 30, 50, … ns). End Time: 1 µs.
- Start = 1 trong khoảng 20–40 ns (nhận tại cạnh lên 30 ns).
- Inport được giữ ổn định cho đến sau cạnh clock kết thúc s1 (≈ 50 ns), vì R1 chỉ nạp Inport tại cạnh này.

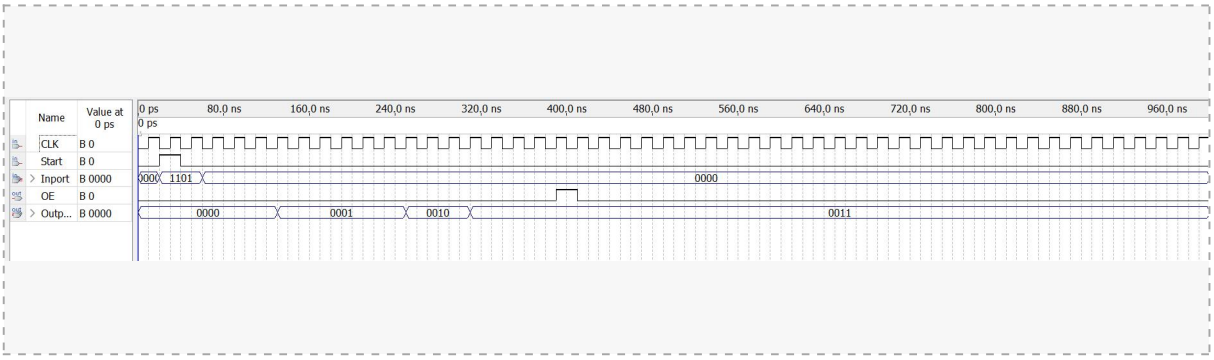
Lưu ý rút ra khi mô phỏng: ở lần chạy đầu, Inport = 1101 chỉ được giữ đến 40 ns (trước cạnh nạp 50 ns), nên R1 nạp 0000 và Outport = 0. Sau khi kéo dài Inport qua cạnh nạp, mạch cho kết quả đúng. Dữ liệu vào mạch đồng bộ phải ổn định tại cạnh clock lấy mẫu.

Bảng 6.1: Các trường hợp kiểm tra

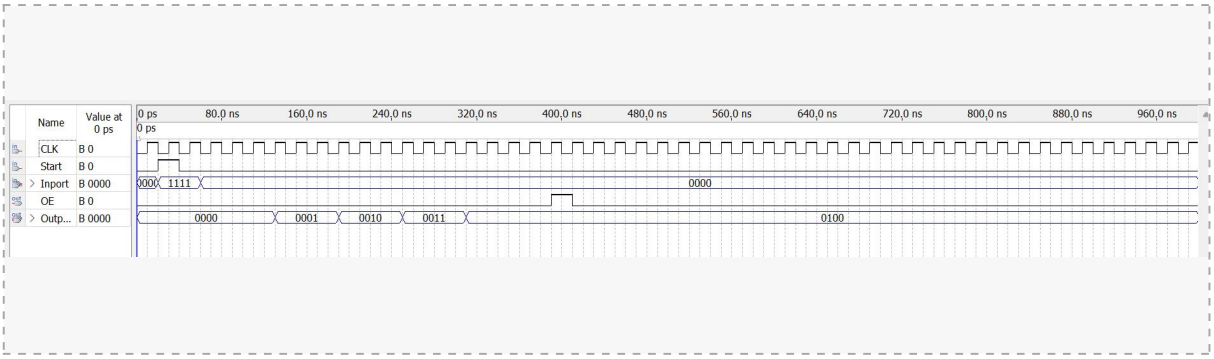
Inport	Số bit 1	Outport mong đợi	Outport mô phỏng	Kết quả
1101	3	0011	0011	Đúng
1111	4	0100	0100	Đúng
0000	0	0000	0000	Đúng
1001	2	0010	0010	Đúng

6.2. Mô phỏng Functional

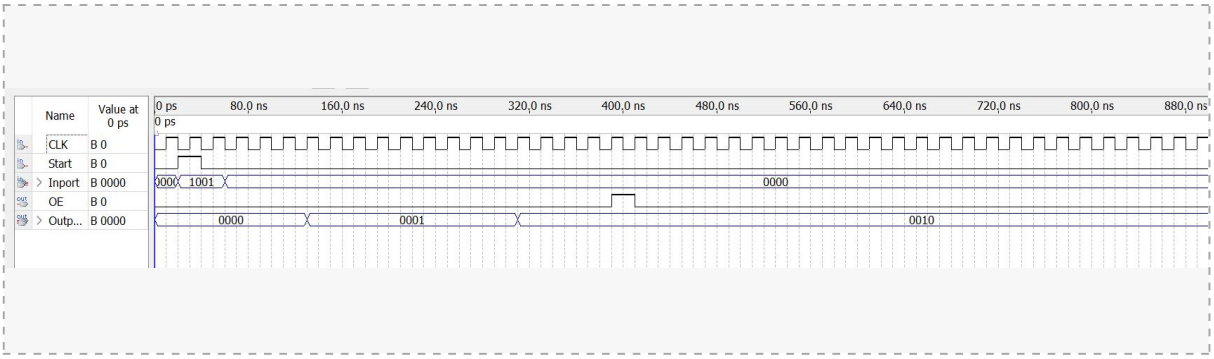
Mô phỏng chức năng bỏ qua độ trễ, dùng để kiểm tra thuật toán và luồng trạng thái.



Hình 6.1: Waveform Functional – Inport = 1101



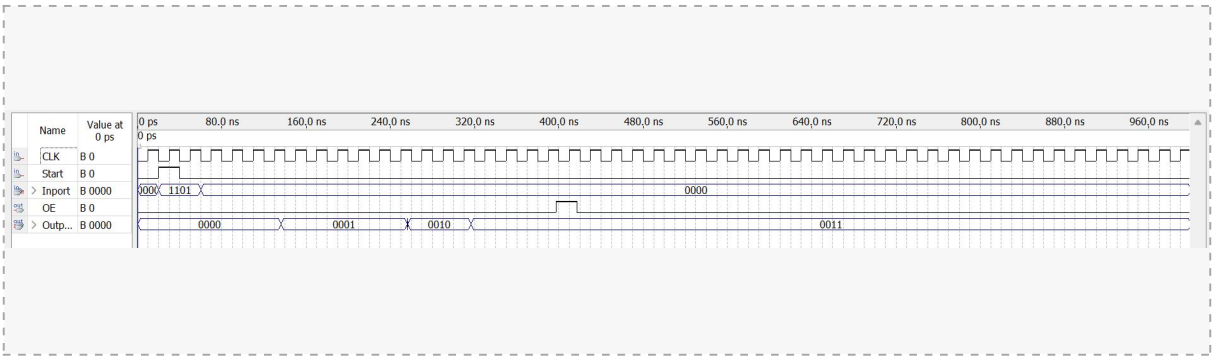
Hình 6.2: Waveform Functional – Inport = 1111



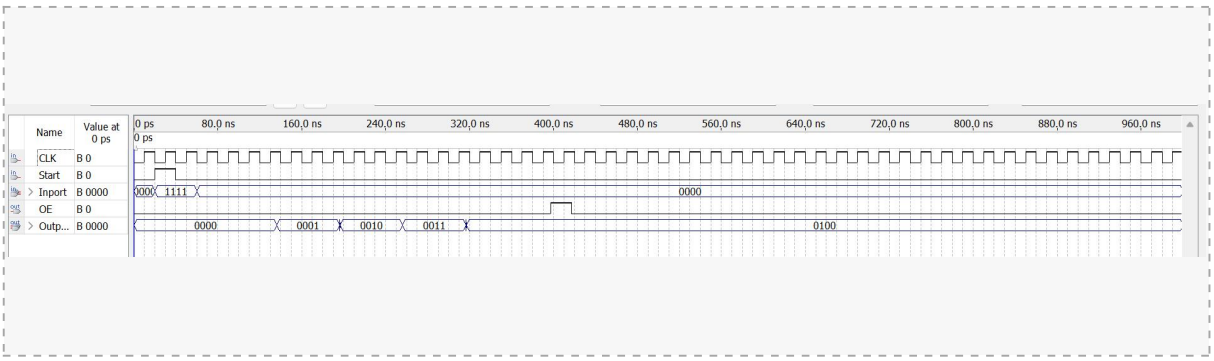
Hình 6.3: Waveform Functional – Inport = 1001

6.3. Mô phỏng Timing

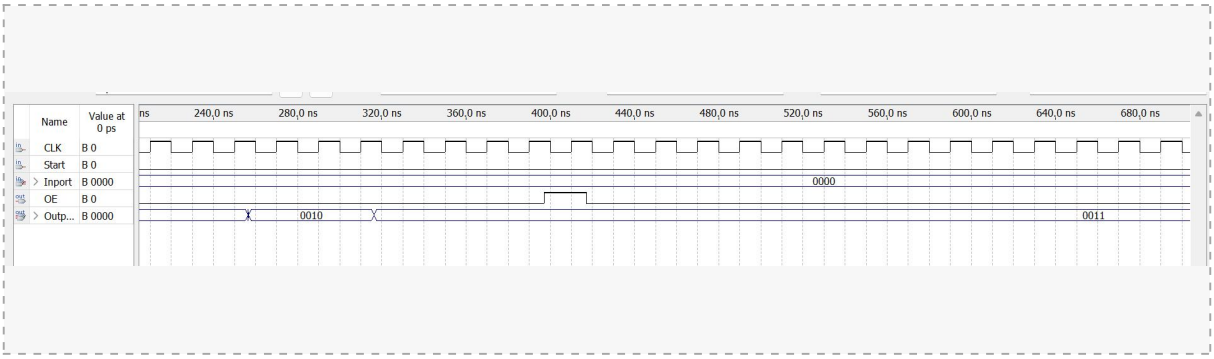
Mô phỏng timing dùng độ trễ thực của cổng logic và dây nối sau bước Fitter, nên tín hiệu ra đổi sau cạnh clock một khoảng và có thể xuất hiện glitch.



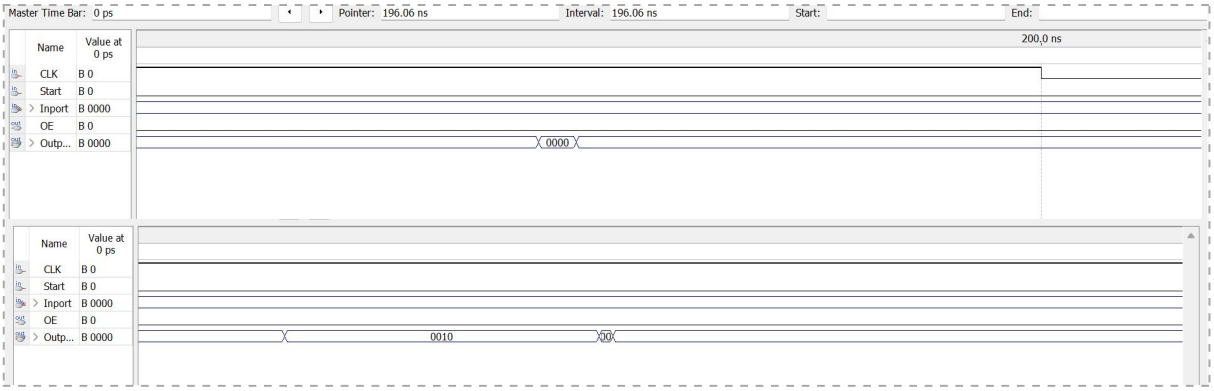
Hình 6.4: Waveform Timing – Inport = 1101



Hình 6.5: Waveform Timing – Inport = 1111



Hình 6.6: Phóng to độ trễ clock-to-output



Hình 6.7: Glitch khi Outport chuyển 1 → 2 và 3 → 4

6.3.1. Độ trễ clock-to-output

Cạnh lên CLK tại ≈ 390 ns, Outport bắt đầu đổi tại ≈ 396 ns → độ trễ $\mathbf{t_{co} \approx 6\ ns}$ (gồm trễ flip-flop và dây nối tới chân ra). Độ trễ này nhỏ hơn nhiều so với chu kỳ 20 ns nên mạch hoạt động đúng ở tần số 50 MHz.

6.3.2. Phân tích glitch

Khi nhiều bit của Outport đổi cùng lúc, mỗi bit có độ trễ đường truyền khác nhau nên xuất hiện giá trị trung gian rất ngắn:

Bảng 6.2: Glitch quan sát được (Inport = 1111)

Chuyển	Nhị phân	Số bit đổi	Giá trị trung gian	Giải thích
0 → 1	0000 → 0001	1	Không	–
1 → 2	0001 → 0010	2	0 (0000)	bit 0 xuống trước bit 1 lên
2 → 3	0010 → 0011	1	Không	–
3 → 4	0011 → 0100	3	2 (0010), rồi 0 (0000)	bit 0 đổi trước, rồi bit 1, cuối cùng bit 2

Glitch không ảnh hưởng kết quả vì hệ thống đồng bộ: các thanh ghi chỉ lấy mẫu tại cạnh clock (khi tín hiệu đã ổn định) và kết quả được đọc khi OE = 1.

6.3.3. So sánh Functional và Timing

Bảng 6.3: So sánh mô phỏng Functional và Timing

Tiêu chí	Functional	Timing
Độ trễ	Bằng 0, đổi đúng tại cạnh clock	Có (≈ 6 ns clock-to-output)
Glitch	Không có	Có khi nhiều bit đổi cùng lúc
Kết quả cuối	1101 → 3; 1111 → 4	1101 → 3; 1111 → 4
Số chu kỳ	19 (1101, 1111)	19 (1101, 1111)
Mục đích	Kiểm tra thuật toán	Kiểm tra khả năng chạy trên chip thật

6.4. Số chu kỳ hoàn thành

Quy ước: tính từ chu kỳ đầu tiên sau khi nhận Start (vào s1) đến hết s7 (OE = 1). Trạng thái chờ s0 không tính.

Bảng 6.4: Diễn biến vòng lặp với Inport = 1101

Vòng	Data (kiểm tra ở s6)	Temp	Ocount sau s5	Data sau dịch	Data = 0?
1	1101	0001	0001	0110	Không → s4
2	0110	0000	0001	0011	Không → s4
3	0011	0001	0010	0001	Không → s4
4	0001	0001	0011	0000	Không → s4
5	0000	0000	0011	0000	Có → s7

Ở s6, điều kiện Data = 0 được kiểm tra bằng giá trị R1 **trước khi dịch** (giá trị dịch chỉ ghi vào R1 tại cạnh clock kết thúc s6), nên vòng lặp chạy thừa 1 lần. Vòng thừa chỉ cộng Temp = 0 nên kết quả vẫn đúng.

Bảng 6.5: Đếm số chu kỳ (Inport = 1101)

Giai đoạn	Trạng thái	Số chu kỳ	Thời gian (T = 20 ns)
Khởi tạo	s1, s2, s3	3	30 – 90 ns
Vòng lặp	5 × (s4, s5, s6)	15	90 – 390 ns
Xuất kết quả	s7	1	390 – 410 ns
Tổng		**19**	**380 ns = 19 × 20 ns**

Công thức tổng quát cho Data 4 bit, với k là vị trí bit 1 cao nhất (bit 0 có k = 0):

$$\textbf{Số chu kỳ} = 3 + 3\cdot(k + 2) + 1 \quad (\text{Data} \neq 0); \quad \text{Data} = 0000\text{: } 7 \text{ chu kỳ}$$

Bảng 6.6: Số chu kỳ theo dữ liệu vào

Inport	Outport	k	Số vòng lặp	Số chu kỳ	Thời gian
0000	0000	–	1	7	140 ns
0001	0001	0	2	10	200 ns
1101	0011	3	5	19	380 ns
1111	0100	3	5	19	380 ns

Nhận xét: số chu kỳ chỉ phụ thuộc vị trí bit 1 cao nhất, không phụ thuộc số lượng bit 1. Ví dụ 1101 và 1111 có kết quả khác nhau nhưng cùng mất 19 chu kỳ. Trường hợp xấu nhất (bit 3 = 1) là 19 chu kỳ.

7. KẾT LUẬN

- Đã thiết kế hoàn chỉnh mạch đếm số bit 1 theo mô hình FSMD trên Quartus: lưu đồ ASM 8 trạng thái, datapath (register file, ALU, shifter, DATA_ZERO) và Control Unit (next-state logic, thanh ghi trạng thái, output logic).
- Mô phỏng Functional và Timing đều cho kết quả đúng: 1101 → 3, 1111 → 4.
- Mô phỏng Timing cho thấy độ trễ clock-to-output ≈ 6 ns và glitch khi nhiều bit đổi cùng lúc; các glitch không ảnh hưởng kết quả do hệ thống đồng bộ.
- Số chu kỳ hoàn thành phụ thuộc dữ liệu: từ 7 chu kỳ (Data = 0) đến tối đa 19 chu kỳ (bit 3 = 1).
- **Ưu điểm:** tiết kiệm phần cứng nhờ dùng chung một ALU và register file. **Nhược điểm:** chậm hơn mạch tổ hợp (cây bộ cộng) do xử lý 1 bit mỗi vòng (3 chu kỳ). Có thể bỏ vòng lặp thừa bằng cách kiểm tra Data = 0 trên giá trị sau khi dịch (tiết kiệm 3 chu kỳ).